

Katha Management App — Documentation

"Katha" (ledger/account book) — a digital ledger app for a salesman who is also a marketer and distributor. It replaces the paper *katha* with a fast, offline-first Flutter app that tracks parties (shops/customers), goods, orders, payments, and a running balance history — and later syncs to the cloud.

1. App Purpose

The client:

- Sells goods to shops/retailers (as a distributor)
 - Markets products (may track promotions/samples)
 - Needs to record: **goods sold, payments received, pending dues, orders placed, orders completed**
 - Needs a searchable **history per customer** (a running ledger, like the paper *katha*)
 - Wants something **fast, offline-capable, good-looking, and able to grow** (more features, more users, cloud sync later)
-

2. Core Features (by priority)

Must-have (MVP)

1. **Party management** — add/edit shops or customers (name, phone, address, area/route, opening balance)
2. **Product/goods catalog** — name, unit, purchase price, sale price, stock qty
3. **Order placement** — create an order for a party with multiple items, quantities, discounts
4. **Order status tracking** — Pending → Placed → Delivered → Cancelled
5. **Payments** — record received/paid amounts, partial payments, payment method (cash/bank/cheque)
6. **Ledger (Katha) view** — chronological debit/credit history per party with running balance
7. **Dashboard** — today's sales, pending dues, recent orders, low stock
8. **Search & filter** — by party, date range, status, product

Should-have (Phase 2)

9. Invoice generation (view/share/print as PDF)
10. Backup/export (CSV/Excel/PDF export of ledger & reports)
11. Multiple price lists / discount rules per party

12. Route/area-based grouping of parties (for daily visit planning)
13. Notifications/reminders for overdue payments

Nice-to-have (Growth phase)

14. Firebase Auth + multi-device cloud sync
 15. Multi-user (owner + salesmen on the field, each with their own login)
 16. Analytics: best-selling products, top customers, monthly trends (charts)
 17. Barcode scanning for products
 18. Offline-first sync engine (local sqflite ↔ Firestore, conflict resolution)
 19. Role-based access (admin vs. field salesman)
 20. Cloud Functions for backups, PDF invoice generation, WhatsApp/SMS reminders
-

3. Data Models

Party (Customer/Shop)

- └─ id (uuid)
- └─ name
- └─ shopName
- └─ phone
- └─ address
- └─ area / route
- └─ type (customer | supplier)
- └─ openingBalance
- └─ currentBalance (derived/cached)
- └─ imageUrl
- └─ isActive
- └─ createdAt
- └─ updatedAt

Product

- └─ id (uuid)
- └─ name
- └─ category
- └─ unit (pcs, kg, box, carton...)
- └─ purchasePrice
- └─ salePrice
- └─ stockQty
- └─ barcode
- └─ imageUrl
- └─ isActive
- └─ updatedAt

Order

- └─ id (uuid)
- └─ orderNumber
- └─ partyId → Party
- └─ orderDate
- └─ deliveryDate
- └─ status (pending | placed | delivered | cancelled)
- └─ items[] → OrderItem
- └─ subtotal
- └─ discount
- └─ totalAmount
- └─ notes

└─ createdBy

OrderItem

└─ id (uuid)

└─ orderId → Order

└─ productId → Product

└─ qty

└─ unitPrice

└─ discount

└─ subtotal

Payment (Transaction)

└─ id (uuid)

└─ partyId → Party

└─ orderId (nullable) → Order

└─ amount

└─ direction (received | paid)

└─ method (cash | bank | cheque | online)

└─ date

└─ notes

└─ createdBy

Invoice

└─ id (uuid)

└─ orderId → Order

└─ invoiceNumber

└─ date

└─ totalAmount

└─ paidAmount

└─ dueAmount

└─ status (unpaid | partial | paid)

KathaEntry (Ledger Line – the heart of the app)

└─ id (uuid)

└─ partyId → Party

└─ date

└─ type (sale | payment | return | adjustment)

└─ refId (orderId or paymentId)

└─ debit (amount party owes, e.g. new sale)

└─ credit (amount party paid)

└─ balanceAfter (running balance snapshot)

└─ note

```
Salesman / User (Phase 3+)
├─ id (uuid)
├─ name
├─ phone
├─ role          (owner | salesman)
├─ assignedArea
└─ isActive
```

Design note: Every sale and every payment writes one `KathaEntry` row. The ledger view is just a query over `KathaEntry` filtered by `partyId`, ordered by date — this is what makes the "history" feel instant and keeps `Party.currentBalance` easy to recompute or cache.

4. Folder Structure

```
lib/
├─ core/
│   ├── constants/      (app_constants.dart, enums.dart)
│   ├── theme/          (app_colors.dart, app_theme.dart, text_styles.dart)
│   ├── utils/          (formatters, validators, date_utils)
│   ├── services/
│   │   ├── db/         (database_helper.dart - sqflite setup, migrations)
│   │   ├── firebase/   (firestore_service.dart, auth_service.dart) [Phase 3]
│   │   └─ sync/        (sync_service.dart) [Phase 3]
│   └─ widgets/         (shared buttons, cards, empty states, loaders)
├─ features/
│   ├── dashboard/
│   ├── parties/        (list, add/edit, detail = ledger view)
│   ├── products/       (list, add/edit, stock)
│   ├── orders/         (list, create, detail, status flow)
│   ├── payments/       (record payment, history)
│   ├── ledger/         (katha view widgets, running balance)
│   ├── invoices/       (generate/view/share) [Phase 2]
│   ├── reports/        (charts, exports) [Phase 2+]
│   ├── settings/       (backup, business profile, printer setup)
│   └─ auth/            [Phase 3]
└─ main.dart
```

Each feature folder internally follows:

```

feature_name/
├─ data/          (repository, local data source)
├─ models/        (or shared in core/models if reused everywhere)
├─ providers/     (or bloc/riverpod state)
├─ presentation/
│   └─ screens/
│   └─ widgets/

```

5. Tech & Architecture Decisions

Concern	Choice	Reason
Local storage	<code>sqflite</code>	Relational data (parties ↔ orders ↔ payments ↔ ledger) needs real joins
State management	Riverpod or Bloc (pick one, be consistent)	Scales cleanly as features grow
Cloud DB (Phase 3)	<code>cloud_firestore</code>	Real-time sync, easy multi-device/multi-user later
Auth (Phase 3)	<code>firebase_auth</code>	Needed once multiple salesmen log in
Repository pattern	Yes	Swap local ↔ remote data source without touching UI code
PDF/Invoice	<code>pdf</code> + <code>printing</code> packages	Share/print invoices
Offline-first sync	Local-first writes, background sync queue	App must work with no internet in the field

Key architectural principle: build a `PartyRepository`, `OrderRepository`, `PaymentRepository` interface now, backed by `SqfliteXRepository` implementations. When Firebase is added, you write a `FirestoreXRepository` implementing the same interface and swap it in — the UI and business logic never change.

6. Development Phases

Phase 0 — Planning & Setup

- Finalize data models & app color/theme (navy + gold, or your chosen palette)
- Set up Flutter project, folder structure, dependencies

- Set up local sqflite database + migrations

Phase 1 — Local MVP (sqflite only)

- Party CRUD
- Product CRUD
- Order creation with multiple items
- Payment recording
- Auto-generated Ledger (Katha) entries on every sale/payment
- Party detail screen = ledger/history view with running balance
- Dashboard: today's sales, total pending, recent activity
- Basic search/filter

Phase 2 — Polish & Local Power Features

- Invoice generation (PDF) + share/print
- Export ledger/reports to CSV/PDF
- Order status workflow (pending → placed → delivered)
- Route/area grouping for parties
- Payment reminders (local notifications)
- Data backup/restore (local file export/import of sqflite db)

Phase 3 — Cloud Migration

- Firebase project setup (FlutterFire CLI, `google-services.json` / `plist`)
- Firebase Auth (owner + salesman roles)
- Mirror sqflite schema into Firestore collections
- Build sync service: local-first write → queue → push to Firestore
- Handle conflict resolution (last-write-wins or timestamp-based)
- Multi-device support or verify offline-first Firestore caching

Phase 4 — Growth Features

- Multi-user roles & permissions (owner sees everything, salesman sees own area)
- Analytics dashboard (best sellers, top customers, monthly trend charts)
- Barcode scanning for products
- WhatsApp/SMS payment reminders (Cloud Functions)
- Cloud-based automated backups

Phase 5 — Scale & Refine

- Performance tuning for large datasets (pagination, indexing)

- Multi-business/tenant support (if the client expands)
- Advanced reporting (profit margins, aging receivables report)
- App polish: animations, empty states, onboarding flow

7. To-Do Checklist

Phase 0 — Setup

```

Init Flutter project, folder structure (core/, features/, main.dart) -----
- [ ]
Add dependencies: sqflite, path_provider, provider/riverpod, intl -----
--[ ]
App theme / AppColors (navy + gold or chosen palette) -----
--[ ]
Set up local database_helper.dart (sqflite init, version, migrations) -----
-- [ ]
Data models: Party, Product, Order, OrderItem, Payment, KathaEntry, Invoice -----
-- [ ]

```

Phase 1 — Local MVP

```

Party repository (CRUD) + Add/Edit Party screen -----
-- [ ]
Product repository (CRUD) + Add/Edit Product screen -----
-- [ ]
Party list screen (search + filter by area) -----
--[ ]
Product list screen (search + stock indicator) -----
--[ ]
Order creation flow (select party → add items → totals) -----
-- [ ]
Order list + status update (pending/placed/delivered/cancelled) -----
-- [ ]
Payment entry screen (received/paid, method, notes) -----
--- [ ]
KathaEntry auto-generation on order + payment save -----
--- [ ]
Party detail screen = Ledger/Katha view (running balance) -----
-- [ ]
Dashboard (today's sales, pending dues, recent orders, low stock) -----
---[ ]

```

Phase 2 — Polish

```
Invoice model + PDF generation -----  
--- [ ]  
Share/print invoice (pdf + printing packages) -----  
--- [ ]  
CSV/PDF export for ledger & reports -----  
---- [ ]  
Route/area grouping UI -----  
----[ ]  
Local notifications for overdue payments -----  
---- [ ]  
Local DB backup/restore (export/import .db file) -----  
---- [ ]
```

Phase 3 — Cloud Migration

```
Firebase project setup (FlutterFire config, google-services.json / plist) -----  
--- [ ]  
Add dependencies: firebase_core, cloud_firestore, firebase_auth -----  
--- [ ]  
Auth screens (login/signup, owner vs salesman role) -----  
---- [ ]  
Firestore schema mirroring sqflite tables -----  
-----[ ]  
Repository interfaces abstracted (Local vs Firestore implementation) -----  
--- [ ]  
Sync service: local-first write + background push to Firestore -----  
--- [ ]  
Conflict resolution strategy implemented -----  
-----[ ]
```

Phase 4 — Growth

```
Role-based permissions (owner/salesman) -----  
----[ ]  
Analytics dashboard (charts: best sellers, top customers, trends) -----  
--- [ ]  
Barcode scanning for products -----  
-----[ ]  
Cloud Function: automated payment reminders (WhatsApp/SMS) -----  
--- [ ]  
Automated cloud backups -----  
-----[ ]
```

Phase 5 — Scale

```
Pagination / indexing for large datasets -----  
-----[ ]  
Multi-tenant support (if needed) -----  
-----[ ]  
Aging receivables report + profit margin report -----  
-----[ ]  
Onboarding flow + UI polish pass -----  
-----[ ]
```

8. Suggested Immediate Next Step

Start Phase 0 exactly as you did before: scaffold the project, lock in the theme, and get the sqlite schema + migrations working for all 7 core models before writing any screens. Getting the `KathaEntry` auto-generation logic right early (every order and payment writes a ledger row) will save you from painful refactors later.